Graded Task 4

- b00094286 - Mohamed Abdelrahim Alawadhi
- b00097405 - Arya Sankhe

In this task, you will fine-tune a pre-trained neural network to train on a custom dataset. Start by downloading train and test example images from 2–3 categories and saving them in a designated folder. Implement a custom dataset class to load these images (refer to this tutorial).

Select AlexNet as your pre-trained model. Replace the final layer with a fully connected layer tailored for your dataset, and freeze all other layers. Fine-tune the model on your dataset and plot the training and testing error for analysis.

```
from torchvision.io import read_image
import torch
from torch.utils.data import Dataset
from torch.utils.data import DataLoader
from torchvision import datasets
from torchvision.transforms import ToTensor
import matplotlib.pyplot as plt
import pandas as pd
from torch import nn
from torchvision import models
from torch import optim
import os


# Load a Pretrained model, weights and the transforms required
from torchvision.models import alexnet
from torchvision.models import AlexNet_Weights
from torchvision import transforms

# Step 1: Initialize model with the weights
weights = AlexNet_Weights.DEFAULT
model = alexnet(weights = weights)
model.eval()

# Step 2: Initialize the inference transforms
preprocess = transforms.Compose([
    transforms.Resize((224,224)),
    transforms.Lambda(lambda x: x.float() / 255.0),
])


# Create a custom Dataset class
class MyImageDataset(Dataset):
    def __init__(self, annotations_file, img_dir, transform=None, target_transform=None):
        self.img_labels = pd.read_csv(annotations_file)
        self.img_dir = img_dir
        self.transform = transform
        self.target_transform = target_transform

    def __len__(self):
        return len(self.img_labels)

    def __getitem__(self, idx):
        img_path = os.path.join(self.img_dir, self.img_labels.iloc[idx, 0])
        image = read_image(img_path)
        label = self.img_labels.iloc[idx, 1]
        if self.transform:
            image = self.transform(image)
        if self.target_transform:
            label = self.target_transform(label)
        return image, label


# Using your custom dataset class load the train and testset

from sklearn.model_selection import train_test_split

dataset = pd.read_csv("/content/annotation.csv")

train, test = train_test_split(dataset, test_size=0.2, random_state=42)

train.to_csv("train.csv", index=False)
test.to_csv("test.csv", index=False)
```

```
trainset = MyImageDataset("/content/train.csv", "/content/Data", preprocess)
batchsize = 2
trainloader = DataLoader(trainset, batch_size=batchsize, shuffle=True)

testset = MyImageDataset("/content/test.csv", "/content/Data", preprocess)
batchsize = 2
testloader = DataLoader(testset, batch_size=batchsize, shuffle=False)


# Define the loss function and the optimizer [see 10-cnn.ipynb]
criteria = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr = 0.001)
train_history = []
val_history = []


# Your Training loop [see 10-cnn.ipynb]
# Training loop
device = torch.device('cuda:0' if torch.cuda.is_available() else 'cpu')

model.to(device)
model.train() # tell the model that your are trainin the model

for epoch in range(10):
    train_loss = 0.0
    for i, data in enumerate(trainloader):
        inputs, labels = data
        inputs = inputs.to(device)
        labels = labels.to(device)

        optimizer.zero_grad()
        outputs = model(inputs)

        loss = criteria(outputs, labels)
        loss.backward()

        optimizer.step()

        train_loss += loss.item()

    # validation
    with torch.no_grad():
        val_loss = 0
        for data in testloader:
            inputs, labels = data
            inputs = inputs.to(device)
            labels = labels.to(device)
            outputs = model(inputs)
            loss = criteria(outputs, labels)
            val_loss += loss.item()

    print(f'Epoch [{epoch}], train loss: {train_loss/len(trainset)}, val loss: {val_loss/len(testset)}')
    train_history += [train_loss/len(trainset)]
    val_history += [val_loss/len(testset)]
print("Finished Training")
```

```
Epoch [0], train loss: 3.8065883219242096, val loss: 0.3631698489189148
Epoch [1], train loss: 4.151008039712906, val loss: 0.2776348292827606
Epoch [2], train loss: 1.1156457513570786, val loss: 0.33520615100860596
Epoch [3], train loss: 0.7074599019251764, val loss: 1.2560272216796875
Epoch [4], train loss: 0.8831183984875679, val loss: 0.755387008190155
Epoch [5], train loss: 0.6862422898411751, val loss: 0.7543782591819763
Epoch [6], train loss: 0.3303053490817547, val loss: 2.3278369903564453
Epoch [7], train loss: 0.5827339738607407, val loss: 5.256626605987549
Epoch [8], train loss: 0.3945266380906105, val loss: 1.6824194192886353
Epoch [9], train loss: 1.4295751275494695, val loss: 10.15285873413086
Finished Training
```

```
# Error analysis [see 10-cnn.ipynb]
import numpy as np

def transform_back(tensor_image):
  image = tensor_image.permute(1, 2, 0).cpu().numpy()
  image = np.clip(image, 0, 1)
  return image

class_to_idx = {"wii": 0, "xbox": 1}
idx_to_class = {0: "wii", 1: "xbox"}
```

```
import torchvision

images, labels = next(iter(testloader))
images = images.to(device)
labels = labels.to(device)

outputs = model(images)
_, predicted = torch.max(outputs, dim=1)

num_images = min(10, images.size(0))

plt.figure(figsize=(20,30))
for i in range(num_images):
    plt.subplot(1,10,i+1)
    plt.tight_layout()
    plt.imshow(transform_back(images[i]))
    plt.axis('off')
    plt.title(idx_to_class[predicted[i].item()])
    print(f"Actual {idx_to_class[labels[i].item()]}\tPredicted: {idx_to_class[predicted[i].item()]}")
plt.show()
```

```
Actual xbox     Predicted: wii
Actual wii      Predicted: wii
```


```
correct = 0
total = 0
with torch.no_grad():
    for data in testloader:
        images, labels = data
        images = images.to(device)
        labels = labels.to(device)
        outputs = model(images)
        _, predicted = torch.max(outputs.data, 1)
        total += labels.size(0)
        correct += (predicted == labels).sum().item()

print(f'Accuracy of the network on the 10000 test images: {100 * correct / total} %%')
```

```
Accuracy of the network on the 10000 test images: 50.0 %%
```

```
# Utilized Chatgpt to evalute the model's performance and visualizes the results of both the training and testing datasets.
model.eval()

all_images = []
all_labels = []
all_predictions = []

with torch.no_grad():
  for images, labels in trainloader:
    images = images.to(device)
    labels = labels.to(device)

    outputs = model(images)
    _, predicted = torch.max(outputs, dim = 1)

    all_images.append(images.cpu())
    all_labels.append(labels.cpu())
    all_predictions.append(predicted.cpu())

with torch.no_grad():
  for images, labels in testloader:
    images = images.to(device)
    labels = labels.to(device)

    outputs = model(images)
    _, predicted = torch.max(outputs, dim = 1)

    all_images.append(images.cpu())
```

```
    all_labels.append(labels.cpu())
    all_predictions.append(predicted.cpu())

all_images = torch.cat(all_images)
all_labels = torch.cat(all_labels)
all_predictions = torch.cat(all_predictions)

num_images = min(100, all_images.size(0))
plt.figure(figsize = (50,40))

for i in range(num_images):
  plt.subplot(10, 10, i + 1)
  plt.imshow(transform_back(all_images[i]))
  plt.axis("off")
  plt.title(f"Actual: {idx_to_class[all_labels[i].item()]}\nPredicted: {idx_to_class[all_predictions[i].item()]}")

plt.show()
```